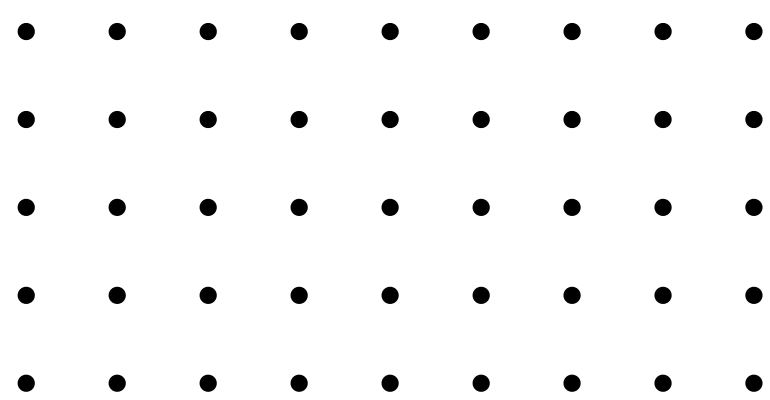
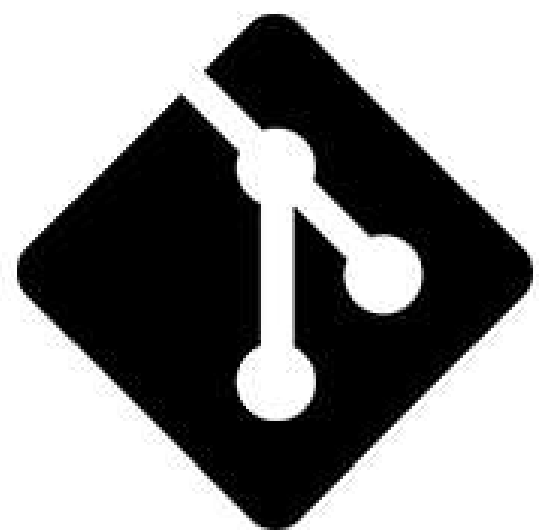




INTRODUÇÃO A GIT E GITHUB



UNIVERSIDADE FEDERAL DE LAVRAS

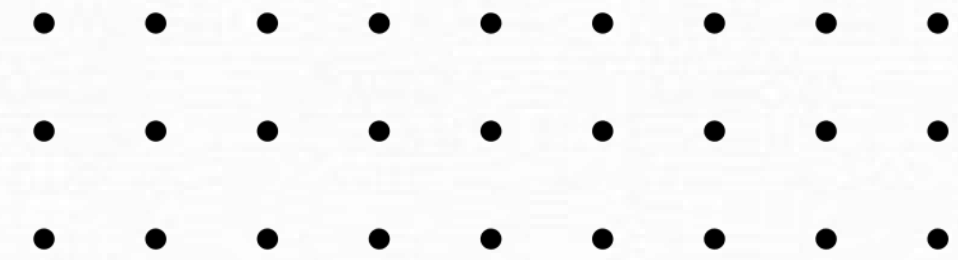
Git ≠ Github



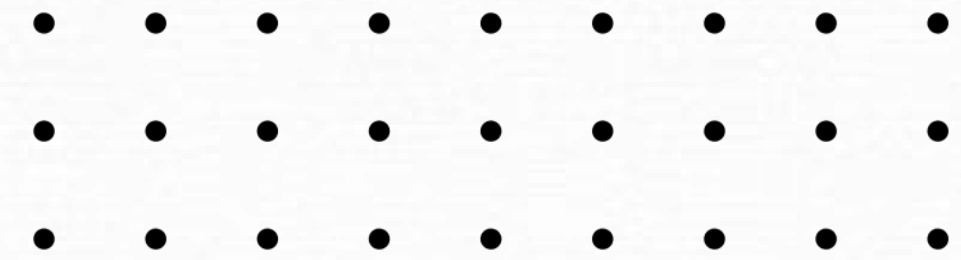
Software de Controle
de Versão



Plataforma de Rede Social
para Programadores



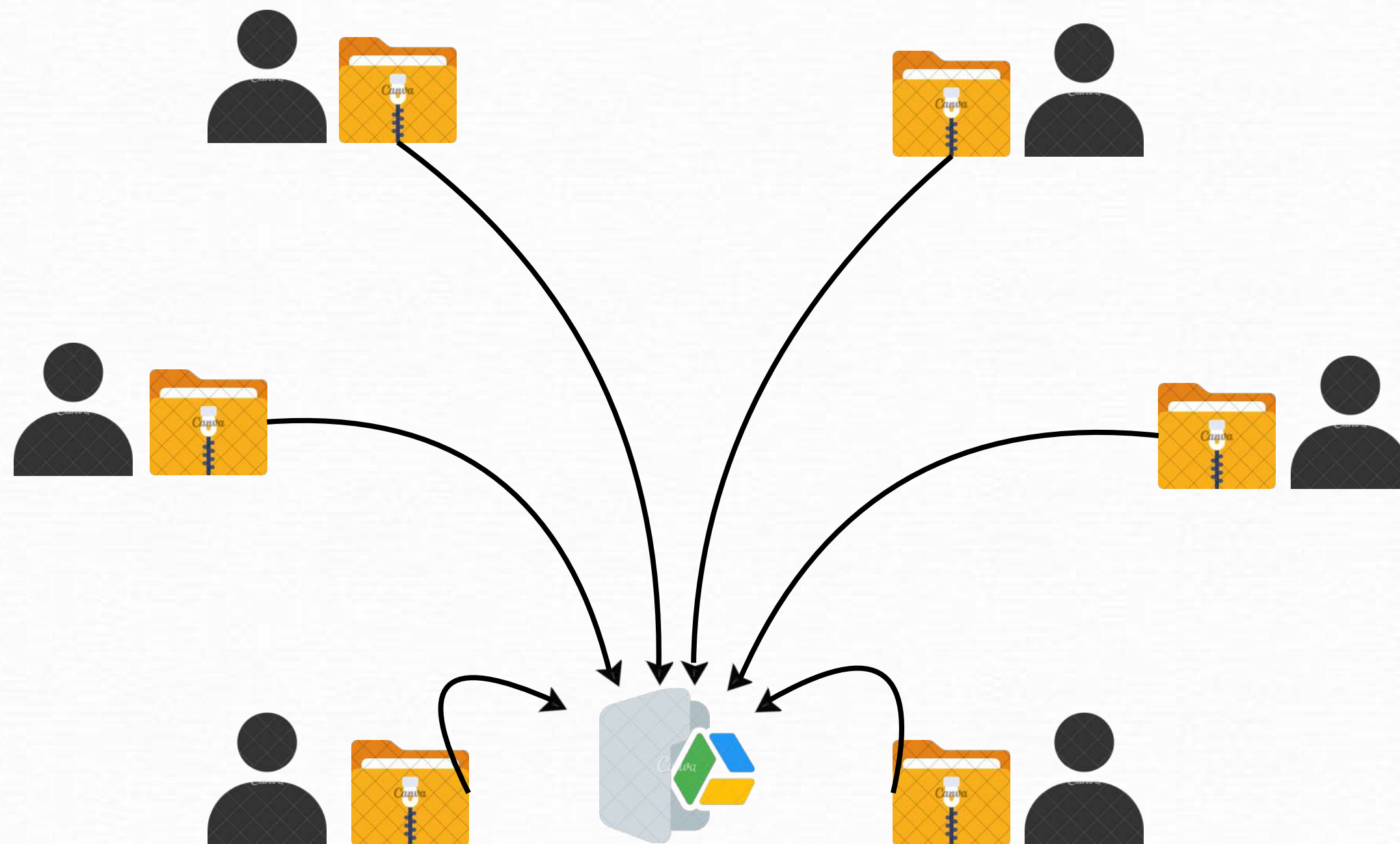
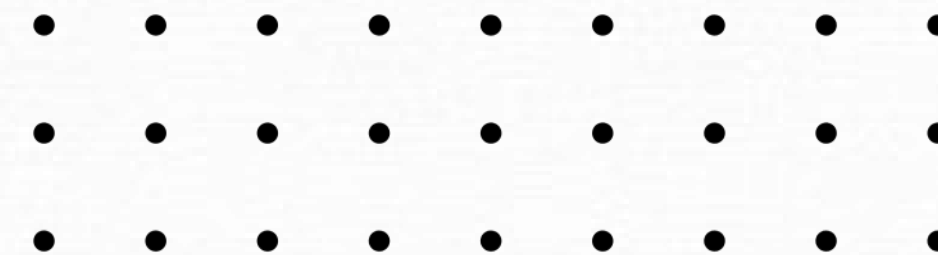
Git



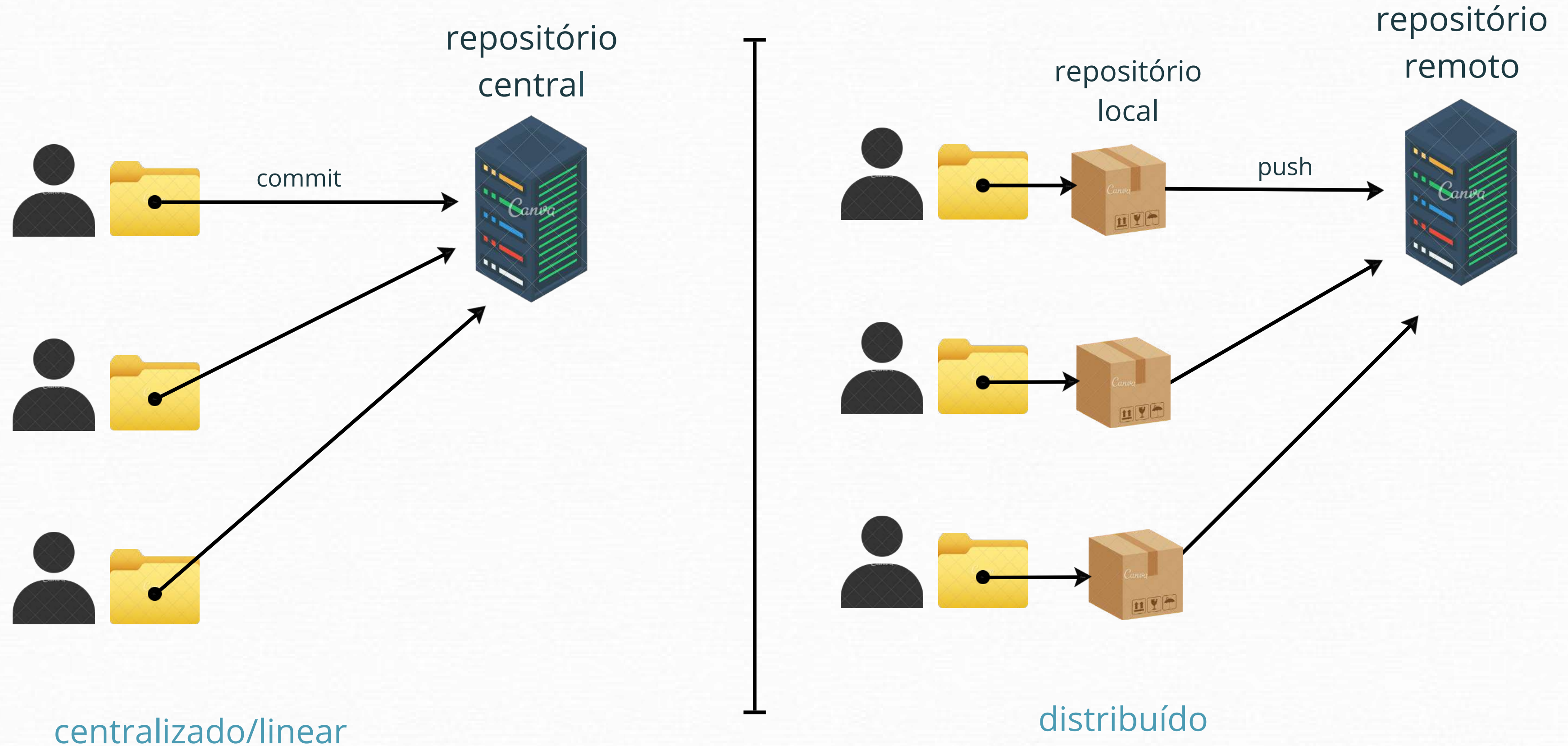
Muitas
versões
de uma
mesma
coisa



Um exemplo



Versionamento (VCS)



Versionamento (VCS)

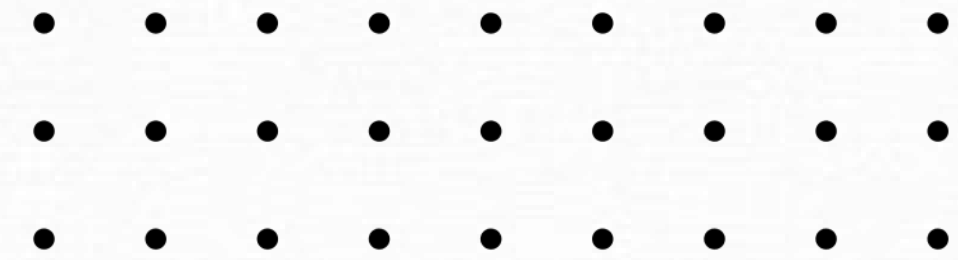
- CA Software Change Manager (CCC)
- Source code Control System (SCCS)
- Panvalet
- Concurrente Version System (CVS)
- Apache Subversion (SVN)
- ClearCase
- Visual SourceSafe
- Perforce

centralizado/linear

- Mercurial
- Bazaar
- Code Co-op
- GNU arch
- Monotone
- Fossil
- BitKeeper
- **Git**

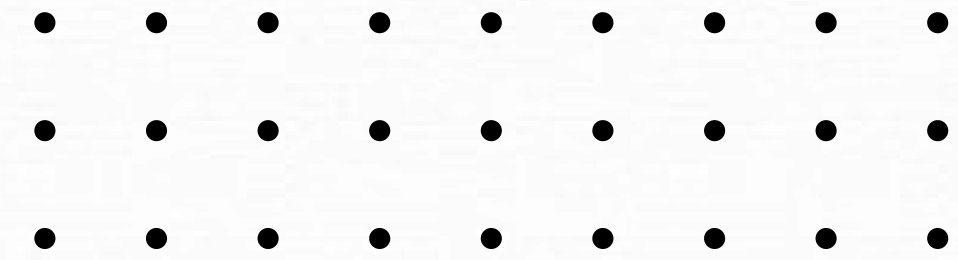
distribuído

Principais vantagens



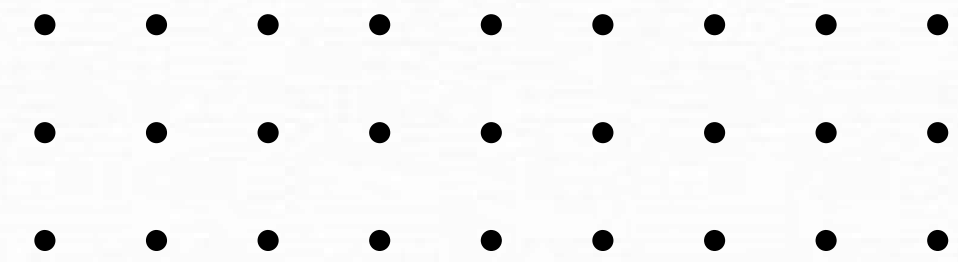
- Controle de histórico
- Trabalho em equipe
- Ramificação do projeto
- Segurança
- Organização

O que é controle de versão?



como você armazenaria diferentes versões do seu código?

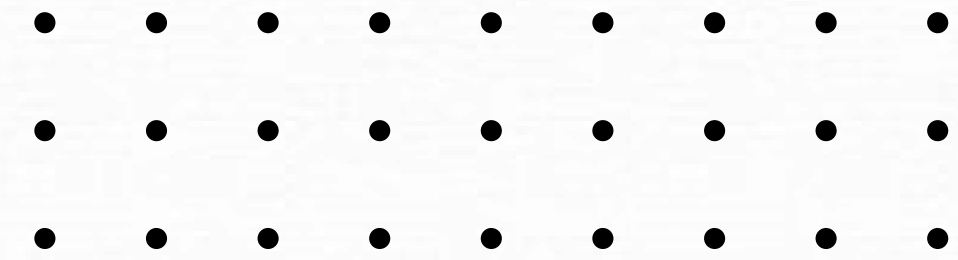
O que é controle de versão?



como você armazenaria diferentes versões do seu código?

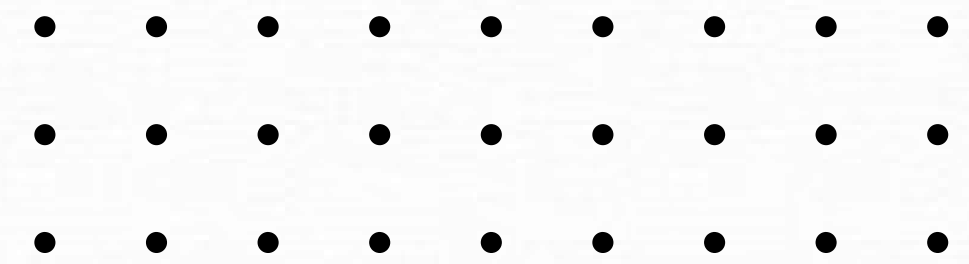
- Criando várias cópias do projeto (Projetov1, Projetov2Final, ProjetoFinalFinal)
- Enviando arquivos por e-mail ou pendrive
- Perdendo alterações importantes ou sobrescrevendo arquivos sem querer

O que é controle de versão?



Uma forma de gerenciar mudanças no código ao longo do tempo.

O que é controle de versão?



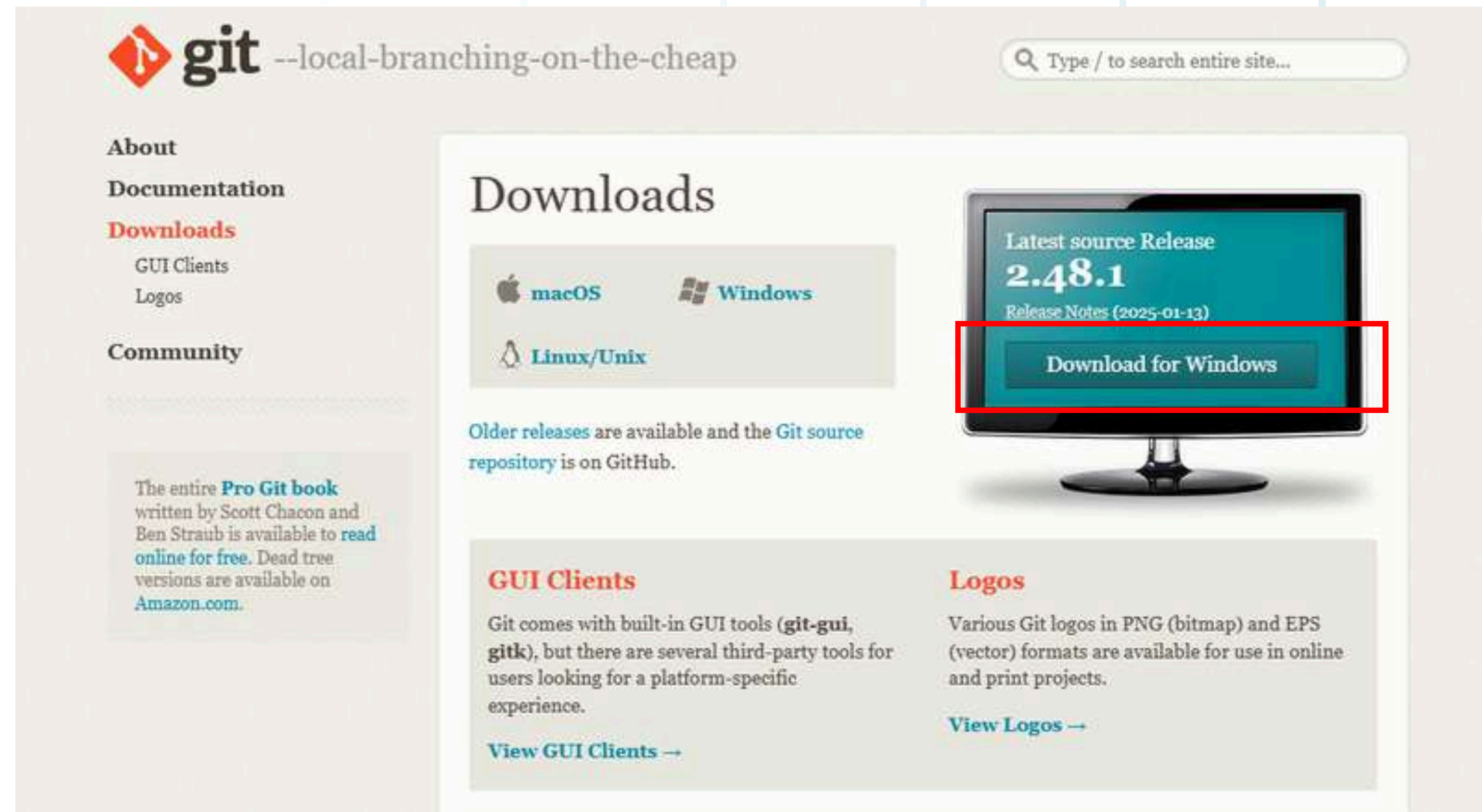
Uma forma de gerenciar mudanças no código ao longo do tempo.

- **Permite salvar versões do projeto sem precisar copiar arquivos**
- **Facilita trabalho em equipe, pois todos têm acesso às mudanças**
- **Ajuda a reverter erros, voltando para versões anteriores quando necessário**

Instalando Git:

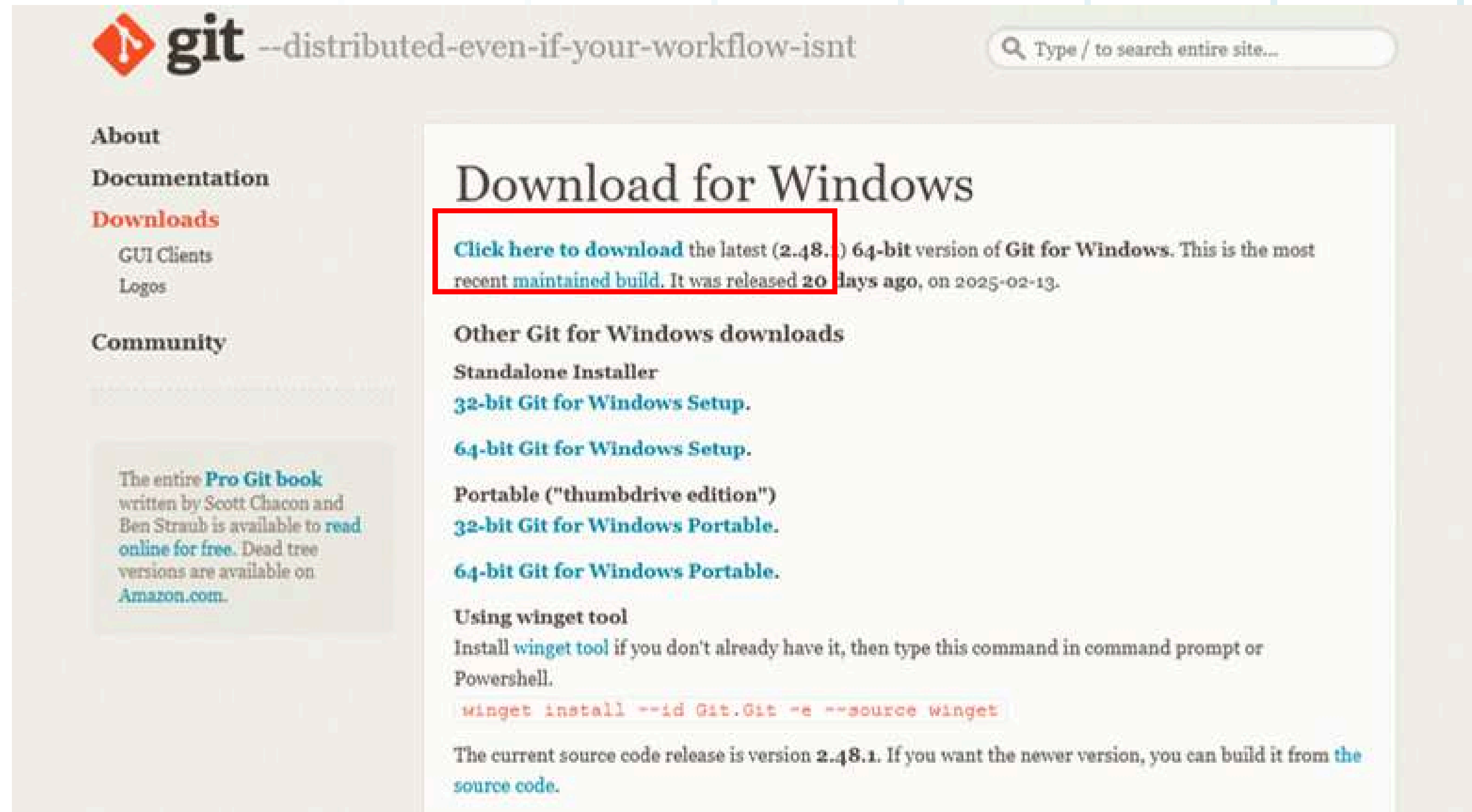
Acesse: <https://git-scm.com/downloads>

Clique em:



Instalando Git:

Clique em:



The screenshot shows the Git website's 'Download for Windows' page. The page has a light beige background with a grid pattern. The Git logo and tagline are at the top. A search bar is in the top right. The left sidebar contains links for 'About', 'Documentation', 'Downloads' (highlighted in red), 'GUI Clients', 'Logos', and 'Community'. The main content area is titled 'Download for Windows'. A red rectangular box highlights the text 'Click here to download' in the first paragraph. Below this, there are sections for 'Other Git for Windows downloads' including 'Standalone Installer', 'Portable ("thumbdrive edition")', and 'Using winget tool'. The 'winget tool' section includes a code block with the command 'winget install --id Git.Git -e --source winget'. At the bottom, it mentions the current source code release is version 2.48.1.

git --distributed-even-if-your-workflow-isnt

Type / to search entire site...

About
Documentation
Downloads
GUI Clients
Logos
Community

The entire **Pro Git book** written by Scott Chacon and Ben Straub is available to read online for free. Dead tree versions are available on Amazon.com.

Download for Windows

[Click here to download](#) the latest (2.48.1) 64-bit version of Git for Windows. This is the most recent **maintained build**. It was released 20 days ago, on 2025-02-13.

Other Git for Windows downloads

Standalone Installer
[32-bit Git for Windows Setup.](#)
[64-bit Git for Windows Setup.](#)

Portable ("thumbdrive edition")
[32-bit Git for Windows Portable.](#)
[64-bit Git for Windows Portable.](#)

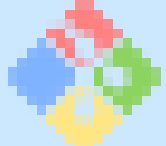
Using winget tool
Install [winget tool](#) if you don't already have it, then type this command in command prompt or Powershell.

```
winget install --id Git.Git -e --source winget
```

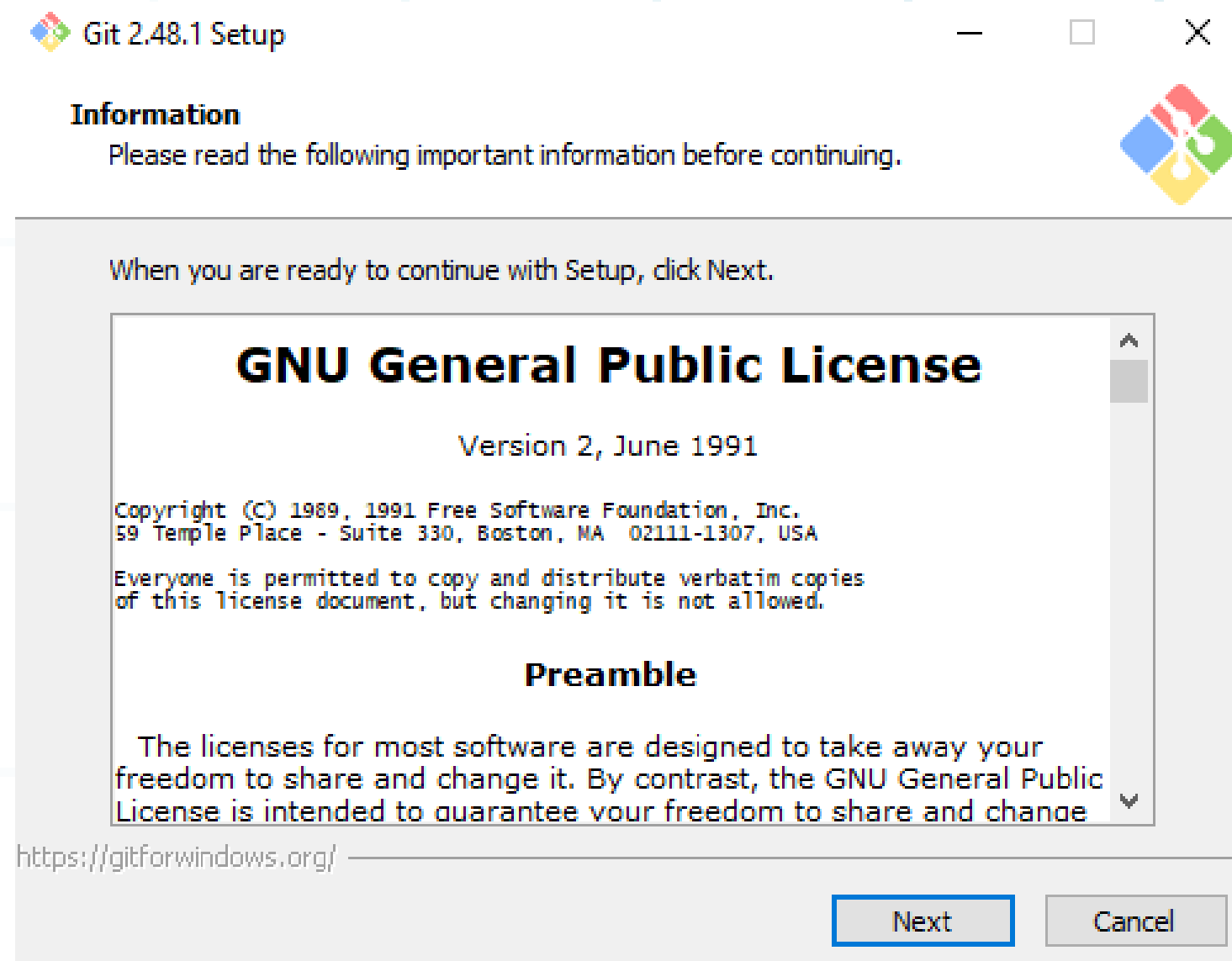
The current source code release is version 2.48.1. If you want the newer version, you can build it from [the source code](#).

Instalando Git:

Execute o arquivo:

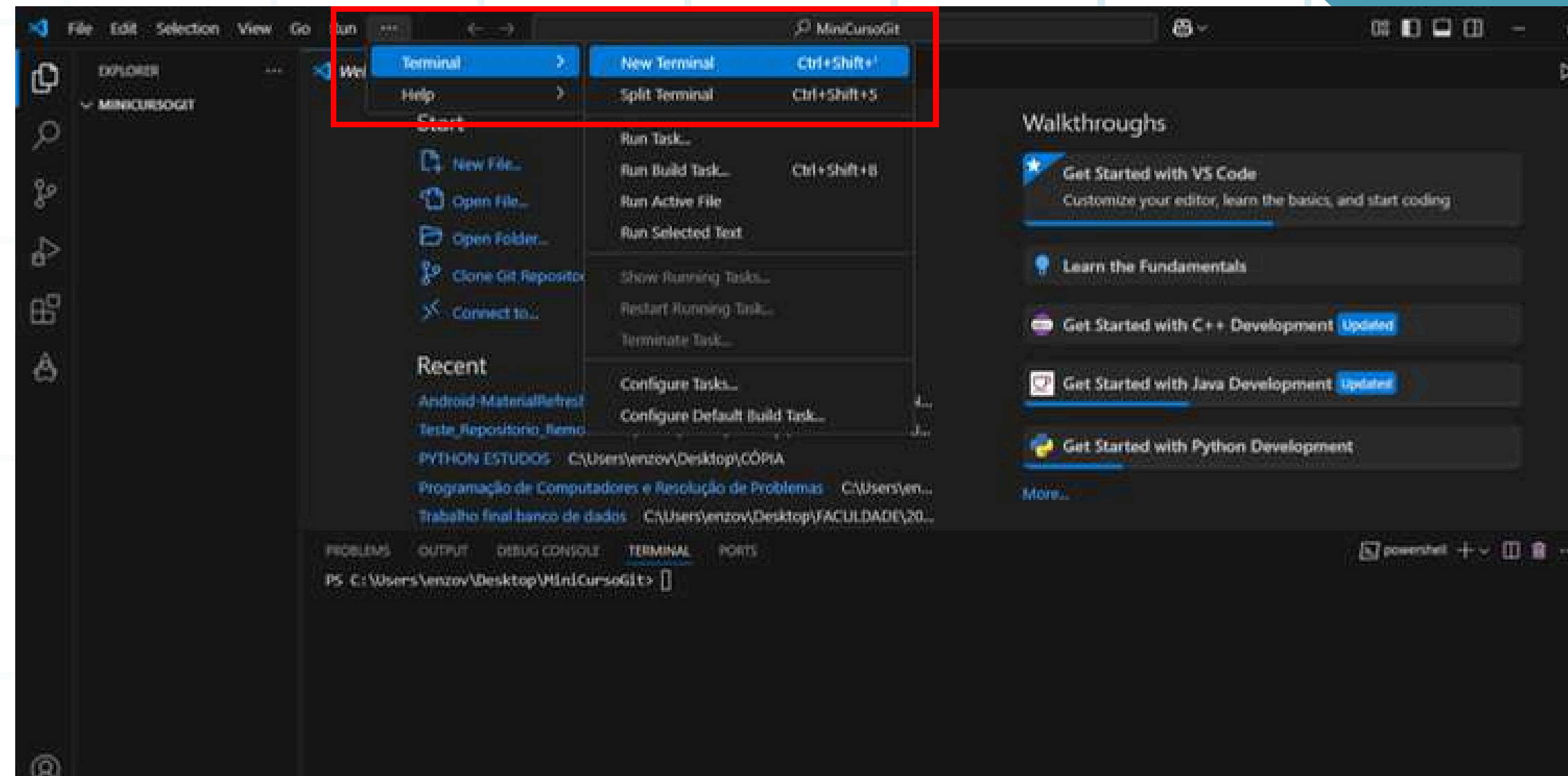
 Git-2.48.1-64-bit

Clique em next até
o final:



Instalando Git:

Entre no VsCode e abra um novo terminal



Instalando Git:

Digite apenas “git” e de enter, caso apareça a seguinte tela a instalação está correta:

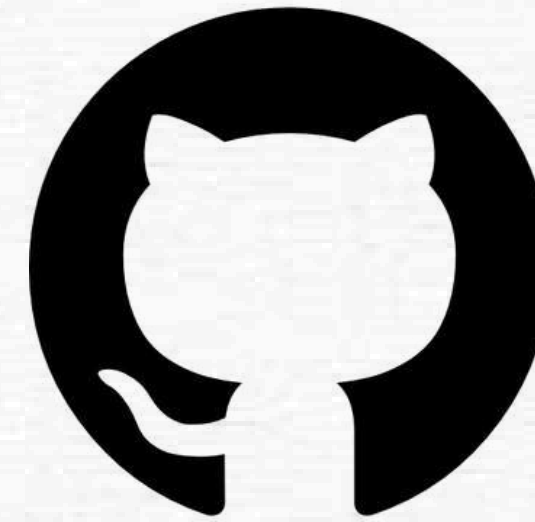
```
PS C:\Users\enzov\Desktop\MiniCursoGit> git
usage: git [-v | --version] [-h | --help] [-C <path>] [-c <name>=<value>]
          [--exec-path[=<path>]] [--html-path] [--man-path] [--info-path]
          [-p | --paginate | -P | --no-pager] [--no-replace-objects] [--no-lazy-fetch]
          [--no-optional-locks] [--no-advice] [--bare] [--git-dir=<path>]
          [--work-tree=<path>] [--namespace=<name>] [--config-env=<name>=<envvar>]
          <command> [<args>]

These are common Git commands used in various situations:

start a working area (see also: git help tutorial)
  clone      Clone a repository into a new directory
  init       Create an empty Git repository or reinitialize an existing one
```

Criando uma conta no GitHub:

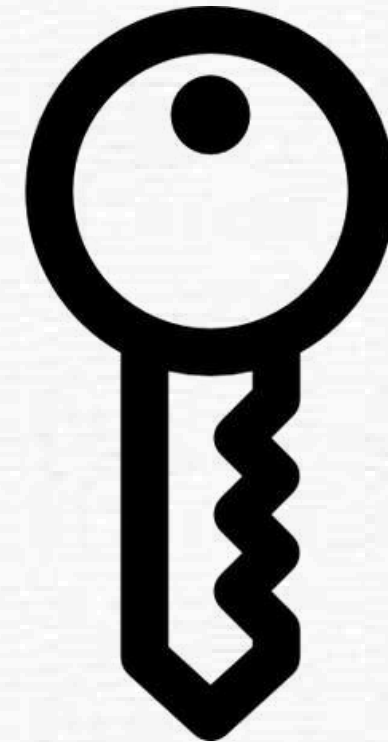
- 1° Acesse: <https://github.com>
- 2° Clique em Sign Up.
- 3° Preencha nome de usuário, e-mail e senha.
- 4° Faça a confirmação no email.



Configurando nosso GitHub para receber nossos commits:

Como o GitHub sabe quem está enviando os commits para o repositório remoto?

Precisamos gerar um token de acesso.



Configurando noss GitHub para receber nossos commits:

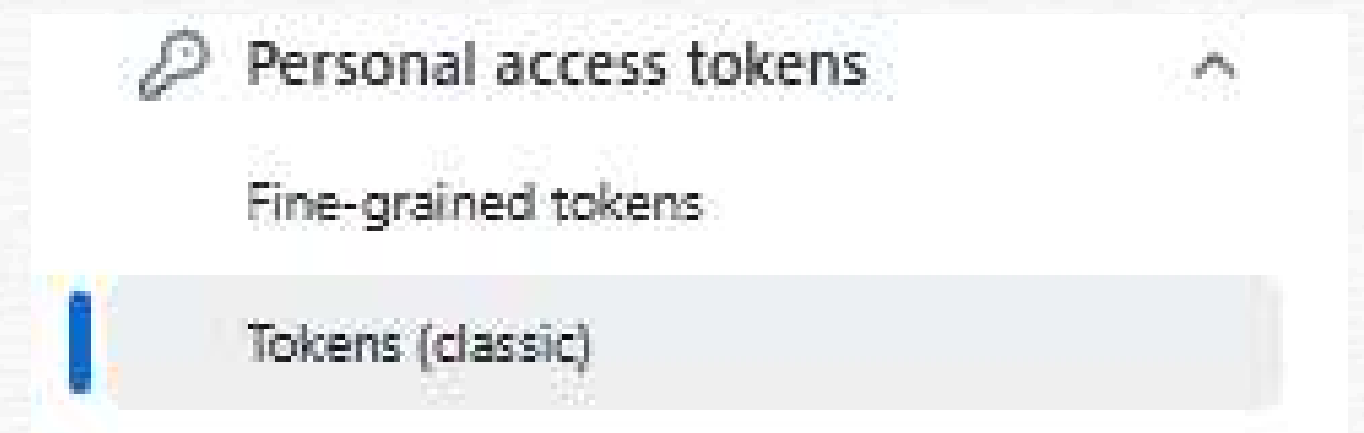
1° Clique no seu perfil e depois em Settings.



2° Clique em Developer settings.



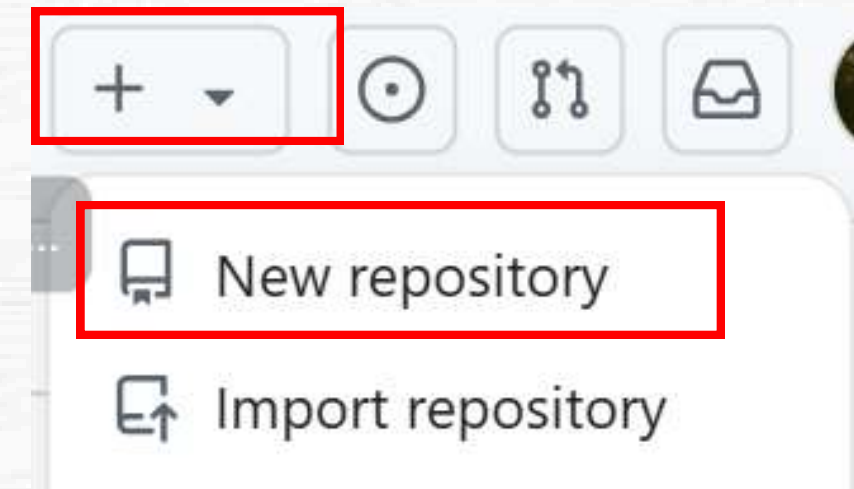
3° Clique em Personal access tokens e Tokens (classic)



Configurando noss GitHub para receber nossos commits:

Criando repositório remoto:

Na pagina inicial do GitHub
clique em create new repository:



Configurando noss GitHub para receber nossos commits:

Criando repositório remoto:

Coloque nome e descrição no seu novo repositório e clique em create:

Required fields are marked with an asterisk (*).

Owner *  enzoavalentim

Repository name *

 RepositorioCursoGit is available.

Great repository names are short and memorable. Need inspiration? How about [miniature-couscous](#) ?

Description (optional)

☒  Public
Anyone on the internet can see this repository. You choose who can commit.

☐  Private
You choose who can see and commit to this repository.

Initialize this repository with:
☐ Add a README file
This is where you can write a long description for your project. [Learn more about READMEs.](#)

Add .gitignore
.gitignore template:

Choose which files not to track from a list of templates. [Learn more about ignoring files.](#)

Choose a license
License:

A license tells others what they can and can't do with your code. [Learn more about licenses.](#)

 You are creating a public repository in your personal account.

Configurando noss GitHub para receber nossos commits:

Removendo configurações antigas:

git config --global --list

git config --global --unset-all user.email

git config --global --unset-all user.name

Configurando noss GitHub para receber nossos commits:

Colocando credenciais:

```
git config --global user.name "Seu Nome de usuario"
```

```
git config --global user.email "seu@email.com"
```

Iniciando o primeiro repositório:

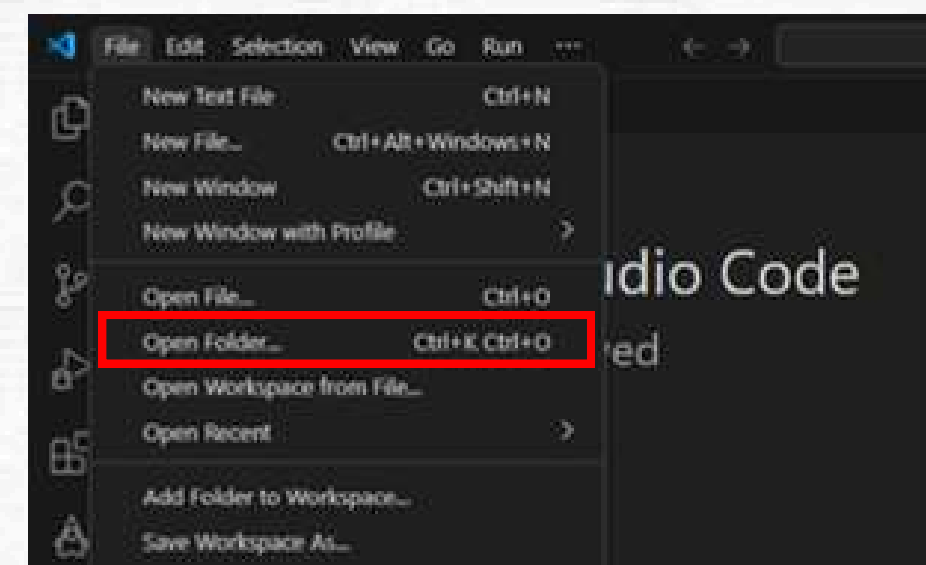
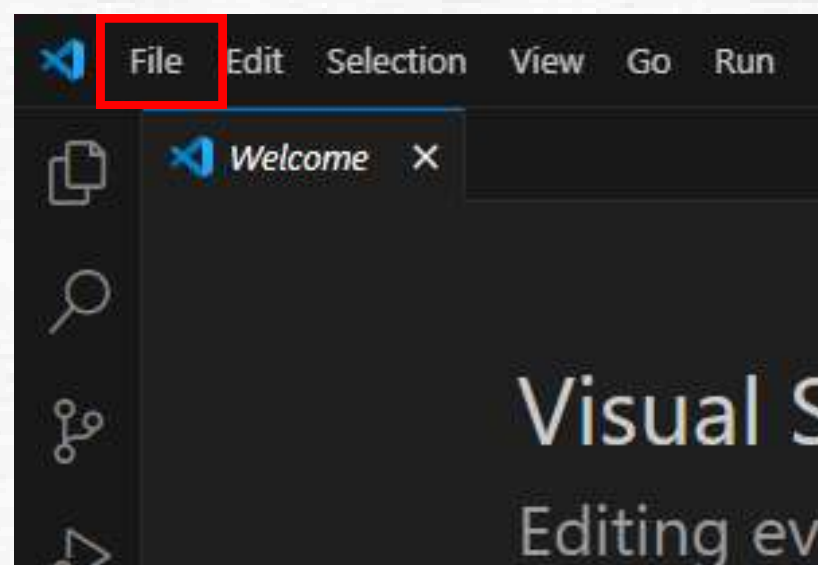
Crie uma pasta na area de trabalho:



MiniCursoGit

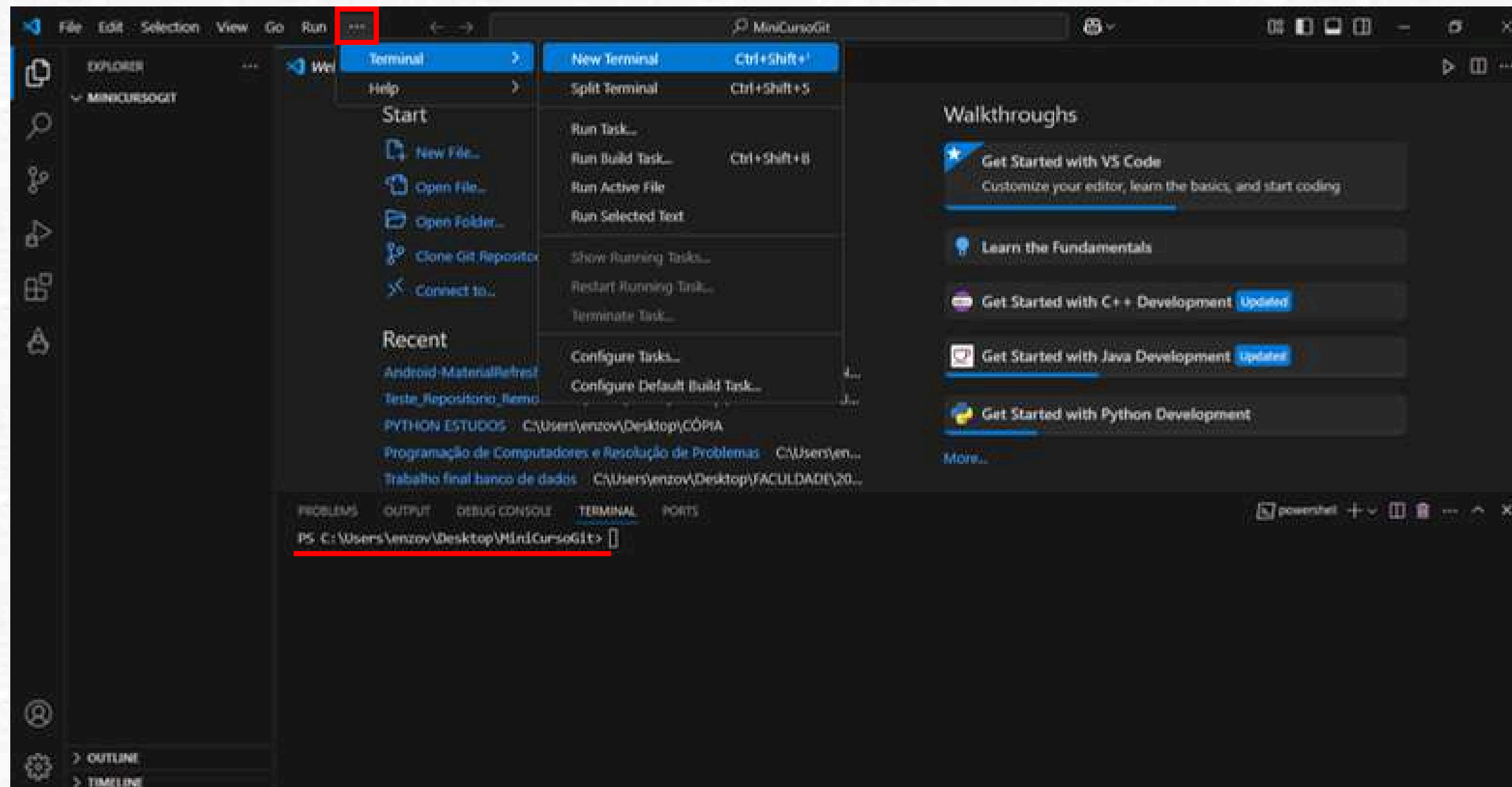
Adicione os arquivos do curso na pasta.

Acesse a pasta pelo VsCode:



Iniciando o primeiro repositório:

Abra um novo terminal no VsCode:

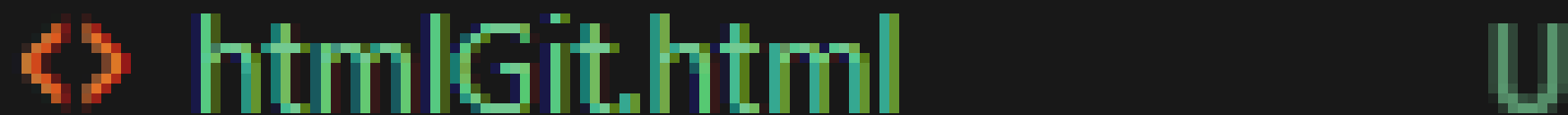


Comandos:

◆ “git init”

- Inicializa um novo repositório Git na pasta atual, permitindo o controle de versão do projeto

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  
PS C:\Users\enzov\Desktop\MiniCursoGit> git init  
Initialized empty Git repository in C:/Users/enzov/Desktop/MiniCursoGit/.git/  
PS C:\Users\enzov\Desktop\MiniCursoGit> 
```



htmlGit.html U

Arquivos que estão **verde** no vscode são arquivos novos que ainda não foram adicionados ao nosso repositório.

O U ao lado do nome do arquivo significa Untracked.

Comandos:

◆ git status

- Exibe o estado atual do repositório, mostrando quais arquivos foram modificados, adicionados ou ainda não rastreados pelo Git.

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

PS C:\Users\enzov\Desktop\MiniCursoGit> git status
On branch master

No commits yet

Untracked files:
  (use "git add <file>..." to include in what will be committed)
    htmlGit.html

nothing added to commit but untracked files present (use "git add" to track)
```

Comandos:

- ◆ **git add <nome do arquivo>**
- ◆ **git add .**

- Adiciona arquivos ao estágio de preparação (staging area), indicando que devem ser incluídos no próximo commit.

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
PS C:\Users\enzov\Desktop\MiniCursoGit> git add htmlGit.html
PS C:\Users\enzov\Desktop\MiniCursoGit> git status
On branch master

No commits yet

Changes to be committed:
  (use "git rm --cached <file>..." to unstage)
    new file:   htmlGit.html
```

htmlGit.html A

O A ao lado do nome do arquivo significa added

Comandos:

◆ `git commit -m "mensagem"`

- Registra as mudanças adicionadas ao repositório, criando um novo ponto no histórico do projeto com uma mensagem descritiva.

OBS: Pense no commit como um checkpoint no seu projeto

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
PS C:\Users\enzov\Desktop\MiniCursoGit> git commit -m "Meu primeiro commit"
[master (root-commit) 0c464e4] Meu primeiro commit
1 file changed, 25 insertions(+)
create mode 100644 htmlGit.html
```

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
PS C:\Users\enzov\Desktop\MiniCursoGit> git status
On branch master
nothing to commit, working tree clean
PS C:\Users\enzov\Desktop\MiniCursoGit> █
```

Comandos:

◆ **git log**

◆ **git log --oneline**

- Exibe o histórico de commits do repositório, mostrando detalhes como autor, data, mensagem e hash de cada commit

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
PS C:\Users\enzov\Desktop\MiniCursoGit> git log
commit 0c464e4bdf791734d7099c0aa358a8bea94c181b (HEAD -> master)
Author: Enzo Augusto Valentim <enzo.valentim@estudante.ufla.com>
Date:   Wed Mar 5 14:07:06 2025 -0300

    Meu primeiro commit
```

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
PS C:\Users\enzov\Desktop\MiniCursoGit> git log --oneline
0c464e4 (HEAD -> master) Meu primeiro commit
```

Altere seu código:

Faça alterações no seu código!

- Imagens
- Formas
- Novas frases



ChatGPT

Gemini



Utilize IAs para ajudar:

Meta AI

Copilot

Altere seu código:

Veja que a cor do seu arquivo mudou:



Arquivos que estão **laranja** são arquivos que foram alterados e não estão commitados ainda, ao lado de arquivos assim também aparecem um M de Modified.

Utilize os comandos que aprendeu para fazer um novo commit.

Comandos:

◆ git diff <hash do commit 1> <hash do commit 2>

- Compara mudanças entre arquivos, commits ou branches, destacando as diferenças entre versões do código. **o que mudou do primeiro commit para o segundo.**

```
PS C:\Users\enzov\Desktop\MiniCursoGit> git log --oneline
58ae78c (HEAD -> master) Adicionando imagem
0c464e4 Meu primeiro commit
```

@@ -31,5 - Na versão antiga do arquivo, o trecho alterado começa na linha 31 e cobre 5 linhas.

@@ +31,8 - Na nova versão, o trecho começa também na linha 31, mas agora são 8 linhas no total.

+linha nova

-linha antiga

```
PS C:\Users\enzov\Desktop\MiniCursoGit> git diff c132b6b eccee6e
diff --git a/htmlGit.html b/htmlGit.html
index 2de89f0..55dd455 100644
--- a/htmlGit.html
+++ b/htmlGit.html
@@ -31,5 +31,8 @@
diff --git a/htmlGit.html b/htmlGit.html
index 2de89f0..55dd455 100644
--- a/htmlGit.html
+++ b/htmlGit.html
@@ -31,5 +31,8 @@
<body>
  <h1>Bem-vindo ao mini curso de Git e GitHub!</h1>
  <p>Altere a pagina como quiser!</p>
+  <div class="image-container">
<body>
<body>
<body>
  <h1>Bem-vindo ao mini curso de Git e GitHub!</h1>
  <p>Altere a pagina como quiser!</p>
+  <div class="image-container">
+    
+  </div>
</body>
</html>
```


Comandos:

◆ **git revert <hash do commit ou HEAD> --no-edit**

- Cria um novo commit voltando com as alterações do hash do commit especificado.

```
PS C:\Users\enzov\Desktop\MiniCursoGit> git log --oneline
eccee6e (HEAD -> master) Adicionando imagem
c132b6b Meu primeiro commit
```

```
PS C:\Users\enzov\Desktop\MiniCursoGit> git revert eccee6e --no-edit
[master 215bce5] Revert "Adicionando imagem"
Date: Thu Mar 6 11:07:26 2025 -0300
1 file changed, 3 deletions(-)
```

DESAFIO: Volte com as suas alterações

Comandos:

◆ **git reset <hash do commit>**

- Move o histórico do repositório para um commit anterior mantendo as alterações feitas no repositório local.

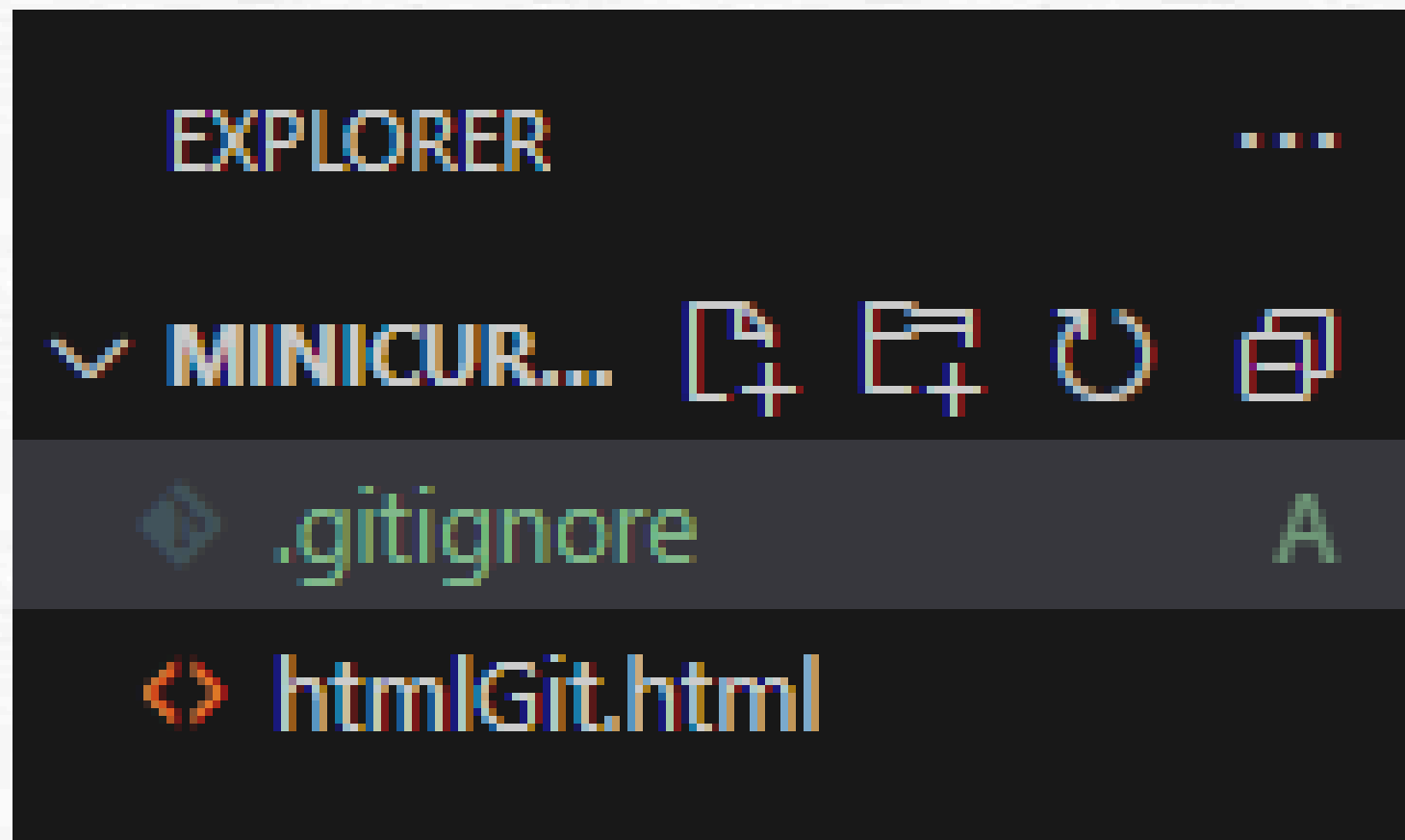
◆ **git stash**

- Salva temporariamente as alterações não commitadas, limpando a área de trabalho sem perder o progresso.

Gitignore:

- Define quais arquivos e pastas devem ser ignorados pelo Git, evitando que sejam rastreados ou versionados no repositório.

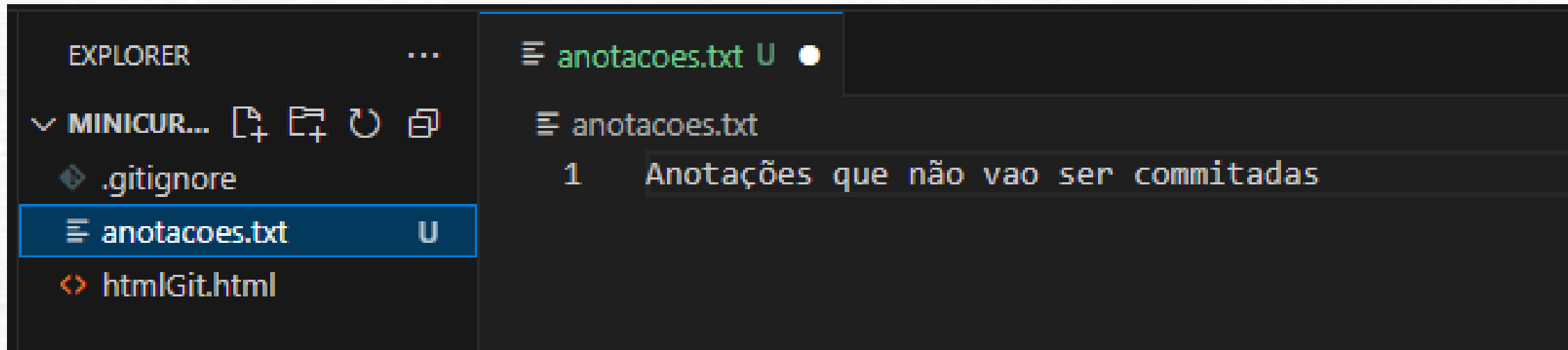
Crie um arquivo no seu repositório com o nome `“.gitignore”` e faça um commit adicionando esse arquivo



```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS
PS C:\Users\enzov\Desktop\MiniCursoGit> git add .gitignore
PS C:\Users\enzov\Desktop\MiniCursoGit> git commit -m "Adicionando gitignore"
[master 8ac3344] Adicionando gitignore
1 file changed, 0 insertions(+), 0 deletions(-)
create mode 100644 .gitignore
```

Gitignore:

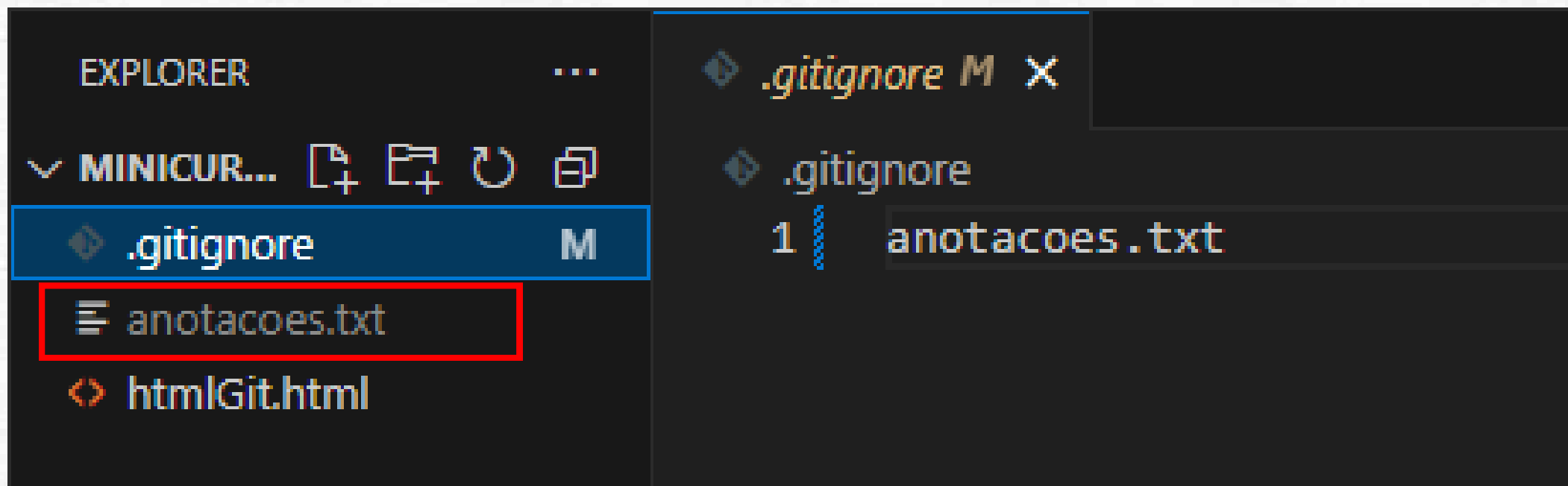
Crie um arquivo .txt



```
EXPLORER
MINICUR...
.gitignore
anotacoes.txt U
htmlGit.html

anotacoes.txt U
1 Anotações que não vao ser commitadas
```

Adicione o nome do arquivo no gitignore.



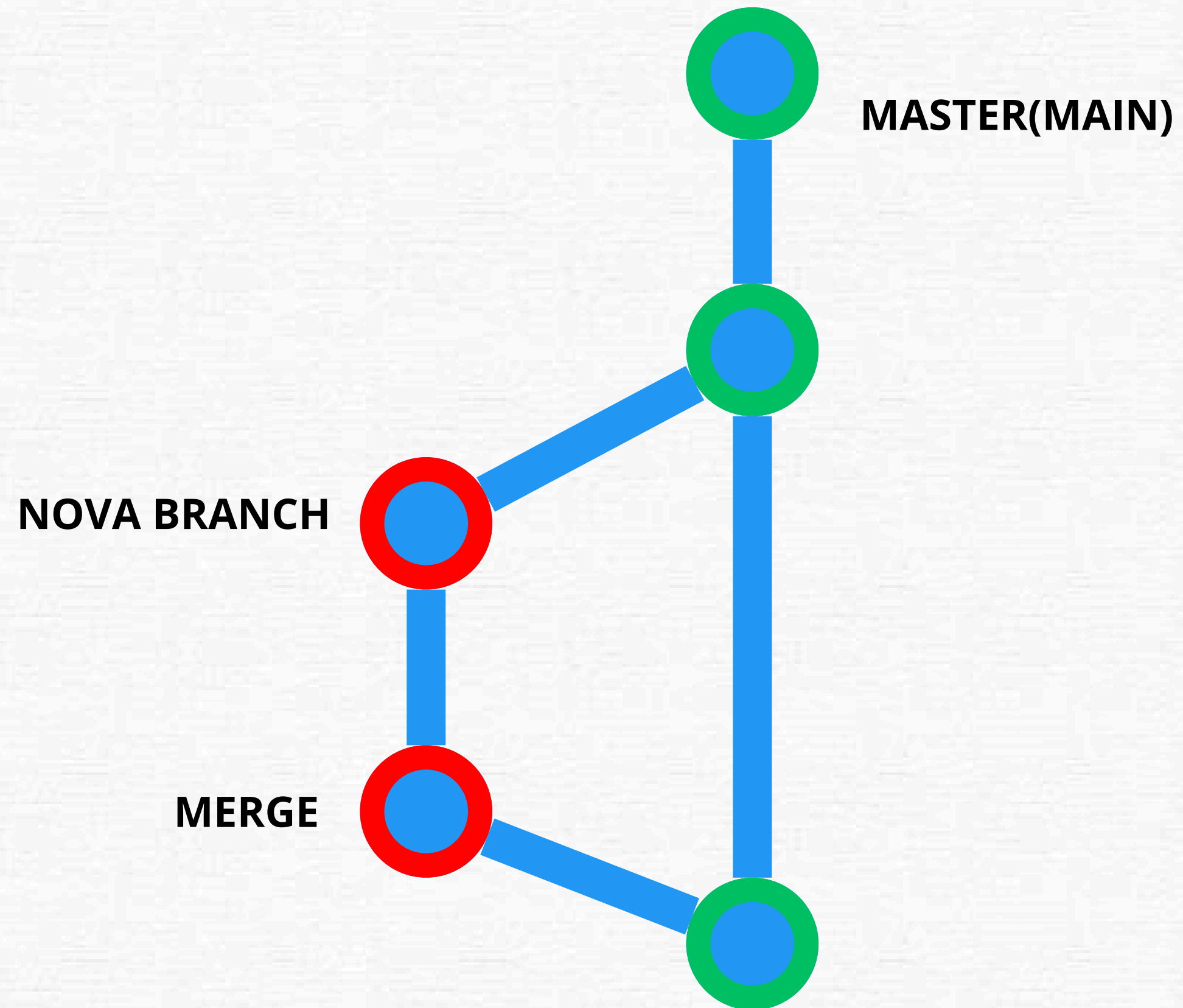
```
EXPLORER
MINICUR...
.gitignore M
anotacoes.txt
htmlGit.html

.gitignore M X
.gitignore
1 | anotacoes.txt
```

O que são Branches no Git?:

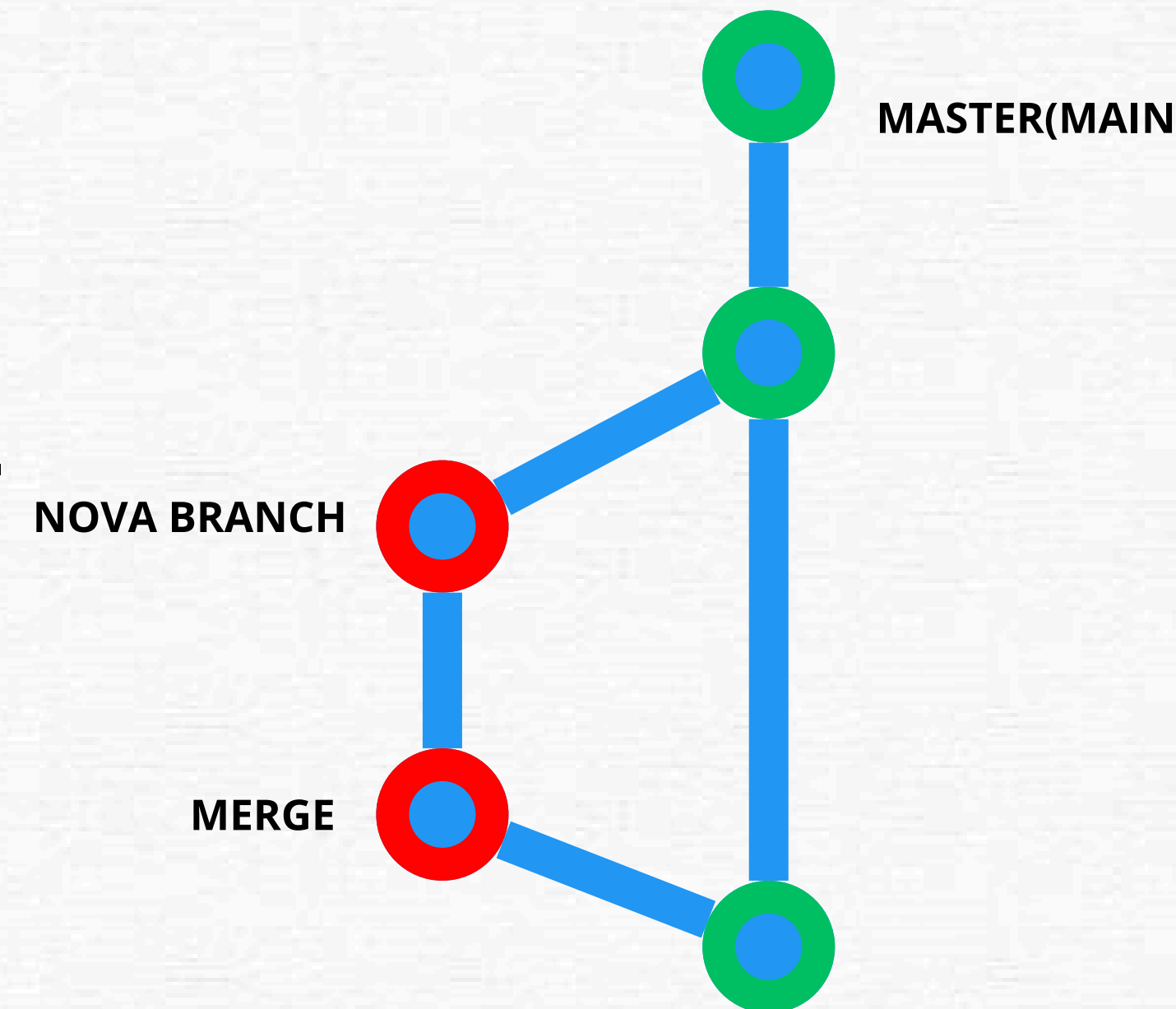
Uma branch (**ramificação**) é uma linha independente de desenvolvimento dentro do repositório.

Ela permite que você trabalhe em novas funcionalidades **sem interferir** no código principal (geralmente a branch main ou master).



Por que usar branches?

- Para desenvolver novas funcionalidades sem bagunçar o código estável.
- Para corrigir bugs isoladamente.
- Para testar ideias sem riscos.
- Para facilitar o trabalho em equipe.



Comandos:

◆ git branch

- O comando git branch lista todas as branches (ramificações) do seu repositório local.

```
PS C:\Users\enzov\Desktop\MiniCursoGit> git branch
* master
```


Comandos:

◆ **git branch <nome-da-branch>**

- Cria uma nova branch a partir da branch atual.

```
PS C:\Users\enzov\Desktop\MiniCursoGit> git branch novaBranch
PS C:\Users\enzov\Desktop\MiniCursoGit> git branch
* master
  novaBranch
```

Comandos:

◆ **git switch <nome-da-branch>**

- Muda para a branch especificada, ou seja, você começa a trabalhar nela.

```
PS C:\Users\enzov\Desktop\MiniCursoGit> git switch novaBranch
Switched to branch 'novaBranch'
PS C:\Users\enzov\Desktop\MiniCursoGit> git branch
master
* novaBranch
```


Altere seu código:

Altere seu código na nova branch!

Faça um commit e volte para a branch principal.

- Imagens
- Formas
- Novas frases

Utilize IAs para ajudar:



ChatGPT

Gemini

Meta AI



Copilot

Altere seu código:

Altere seu código na nova branch!

Faça um commit e volte para a branch principal.

```
PS C:\Users\enzov\Desktop\MiniCursoGit> git add .  
PS C:\Users\enzov\Desktop\MiniCursoGit> git commit -m "Aumentando tamanho das letras"  
[novaBranch bfd26c7] Aumentando tamanho das letras  
1 file changed, 3 insertions(+)  
PS C:\Users\enzov\Desktop\MiniCursoGit> git switch master  
Switched to branch 'master'
```

Percebeu algo?

Comandos:

◆ **git merge <nome da branch>**

- Usado para unir o histórico de uma branch com outra. Ele incorpora as alterações feitas em uma branch (por exemplo, uma feature) na branch atual.

```
PS C:\Users\enzov\Desktop\MiniCursoGit> git merge novaBranch
Updating 8518532..bfd26c7
Fast-forward
 htmlGit.html | 3 +++
1 file changed, 3 insertions(+)
```

Comandos:

◆ **git branch -D <nome da branch>**

- Usado para excluir um branch local.

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

PS C:\Users\enzov\Desktop\MiniCursoGit> git branch -D novaBranch
Deleted branch novaBranch (was e0123cd).
PS C:\Users\enzov\Desktop\MiniCursoGit> █
```

Conflitos entre branches:

O que é um conflito no Git?

- Um conflito de merge acontece quando o Git não consegue automaticamente combinar mudanças feitas em diferentes branches, porque dois commits modificaram a mesma parte do mesmo arquivo.

```
16  */
17  Accept Current Change | Accept Incoming Change | Accept Both Changes | Compare Changes
18  <<<<<<< HEAD (Current Change)
19  function printMessage(showUsage, message) {
20      console.log(message);
21  }
22  =====
23  function printMessage(showUsage, showVersion) {
24      console.log("Welcome To Line Counter");
25      if (showVersion) {
26          console.log("Version: 1.0.0");
27      }
28  }
29  >>>>>>> theirs (Incoming Change)
30  if (showUsage) {
31      console.log("Usage: node base.js <file1> <file2> ... ");
32  }
```

← **Versão da branch atual**

← **Versão da branch que está sendo mesclada**

Resolvendo Conflitos entre branches:

- 1º Crie e selecione uma nova branch;
- 2º Altere uma linha;
- 3º Faça um commit com a linha alterada.

```
36     <p>Esta frase foi editada na branch branchDeConflito.</p>
37     <div class="image-container">
38          
```

Resolvendo Conflitos entre branches:

4° Volte a branch principal;

5° Altere a mesma linha do commit anterior;

6° Faça um commit com a linha alterada.

```
36     <p>Esta frase foi editada na branch master.</p>
37     <div class="image-container">
38         
40 </body>
41 </html>
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\enzov\Desktop\MiniCursoGit> git branch
branchDeConflito
```

```
* master
```

```
PS C:\Users\enzov\Desktop\MiniCursoGit> git add htmlGit.html
```

```
PS C:\Users\enzov\Desktop\MiniCursoGit> git commit -m "Alterando a branch master"
```

```
[master 695d212] Alterando a branch master
```

```
1 file changed, 1 insertion(+), 1 deletion(-)
```


Resolvendo Conflitos entre branches:

- 7° Ainda na branch principal faça um merge com a nova branch;
- 8° Edite manualmente o conteúdo do código e realize um commit ou utilize Merge editor do vsCode.

```
36 <<<<<< HEAD (Current Change)
37 <p>Esta frase foi editada na branch master.</p>
38 =====
39 <p>Esta frase foi editada na branch branchDeConflito.</p>
40 >>>>>> branchDeConflito (Incoming Change)
41 <div class="image-container">
42 |   
6. git push -u origin main

```
>> git init
>> git add README.md
>> git commit -m "first commit"
>> git branch -M main
>> git remote add origin https://github.com/Bytsuki0/teste.git
>> git push -u origin main
Initialized empty Git repository in D:/Code/test/.git/
[master (root-commit) bd45a29] first commit
1 file changed, 0 insertions(+), 0 deletions(-)
create mode 100644 README.md
Enumerating objects: 3, done.
Counting objects: 100% (3/3), done.
Writing objects: 100% (3/3), 241 bytes | 241.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0
To https://github.com/Bytsuki0/teste.git
 * [new branch] main -> main
branch 'main' set up to track 'origin/main'.
```

# Trabalhando com um repositório online

**O terceiro modo é adicionando um repositório local a um espaço na nuvem:**

1. `git remote add origin https://github.com/Usuario/Repositório.git`
2. `git branch -M main`
3. `git push -u origin main`

**Caso você possua um repositório local e apenas queira que ele seja adicionando a um endereço na nuvem, esse é o comando que deve se utilizar.**



# **Configurando nosso GitHub para receber nossos commits:**

## **Linkando repositório local ao remoto:**

**No terminal do VsCode e digite os seguintes comandos:**

```
git remote add origin https://github.com/usuario/nome-repo.git
```

**O link do repositório aparece em sua pagina do GitHub.**

# Comandos:

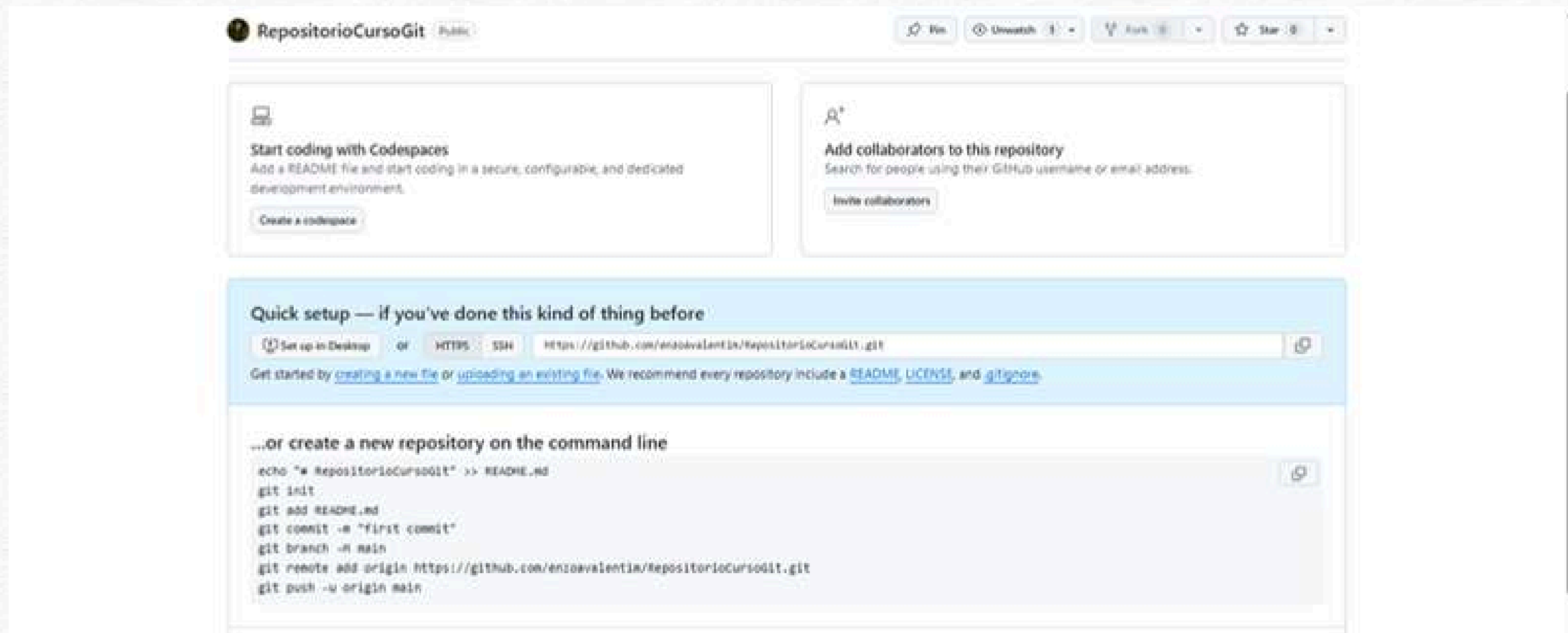
- ◆ **git branch -M main**
- ◆ **git push -u origin main**
- Envia os commits locais para o repositório remoto associado, normalmente na branch padrão.

```
PS C:\Users\enzov\Desktop\MiniCursoGit2> git remote add origin https://github.com/enzoavalentim/RepositorioCursoGit.git
PS C:\Users\enzov\Desktop\MiniCursoGit2> git push origin
Enumerating objects: 3, done.
Counting objects: 100% (3/3), done.
Delta compression using up to 8 threads
Compressing objects: 100% (2/2), done.
Writing objects: 100% (3/3), 630 bytes | 210.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/enzoavalentim/RepositorioCursoGit.git
 * [new branch] master -> master
branch 'master' set up to track 'origin/master'.
```



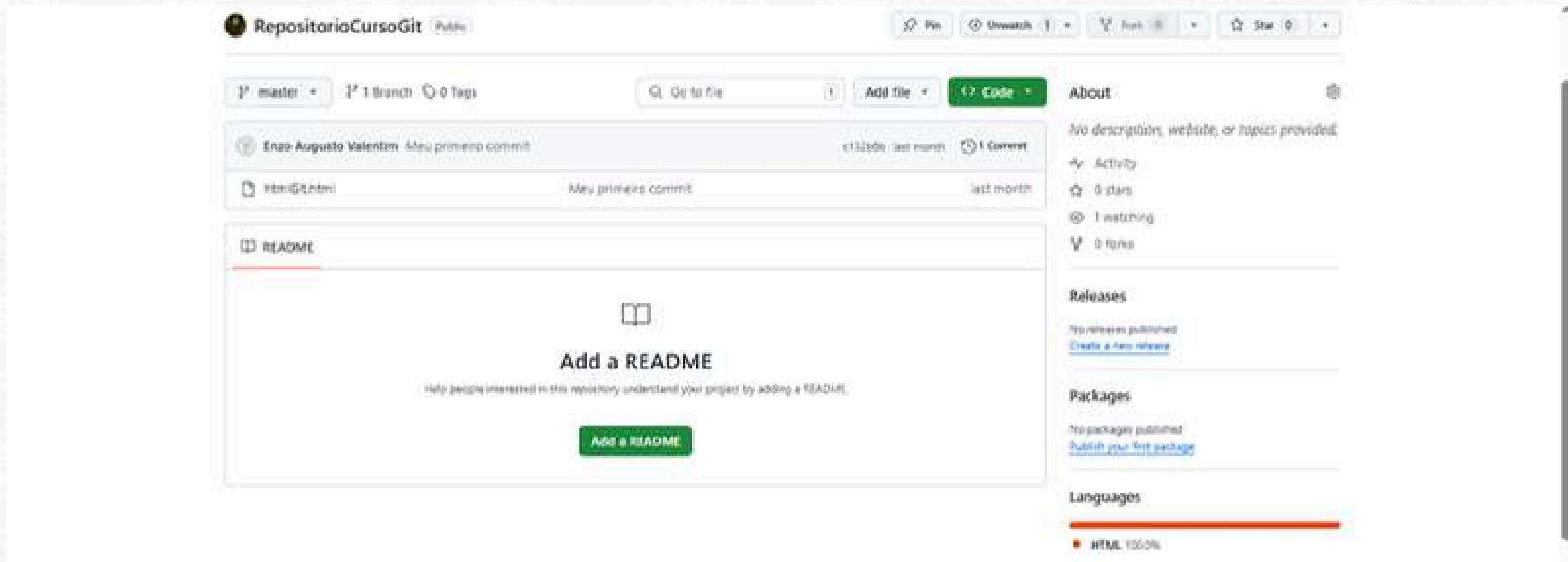
# Comandos:

## ◆ git push



# Comandos:

## ◆ git push



# Comandos:

## ◆ git fetch origin

- Obtém todas as informações no repositório remoto (branch atual do hub) e deixa elas salvas em cache para atualizar o repositório local

**OBS: Você estará simplesmente atualizando seu repositório local com o remoto**

```
PS D:\Code\Python> git fetch origin
PS D:\Code\Python> git fetch origin main
From https://github.com/Bytsuki0/Python
* branch main -> FETCH_HEAD
PS D:\Code\Python>
```

```
PS D:\Code\Python> git status
On branch main
Your branch is behind 'origin/main' by 1 commit, and can be fast-forwarded.
(use "git pull" to update your local branch)
```

# Comandos:

## ◆ `git pull origin --branch*`

- Seu repositório local será trocado pelo repositório remoto atualizando as informações em cache do fetch

**OBS: Você troca seus arquivos locais pelo atual branch remoto (branch pode ser qualquer partição do seu repositório main por exemplo)**

```
PS D:\Code\Python> git pull origin main
From https://github.com/Bytsuki0/Python
 * branch main -> FETCH_HEAD
Updating 4f8a71c..357db5f
Fast-forward
 IC_git_blame/git_blameTemp.py | 2 +-
 IC_git_blame/gitblame.py | 2 +-
 2 files changed, 2 insertions(+), 2 deletions(-)
PS D:\Code\Python>
```

```
PS D:\Code\Python> git status
On branch main
Your branch is up to date with 'origin/main'.
```

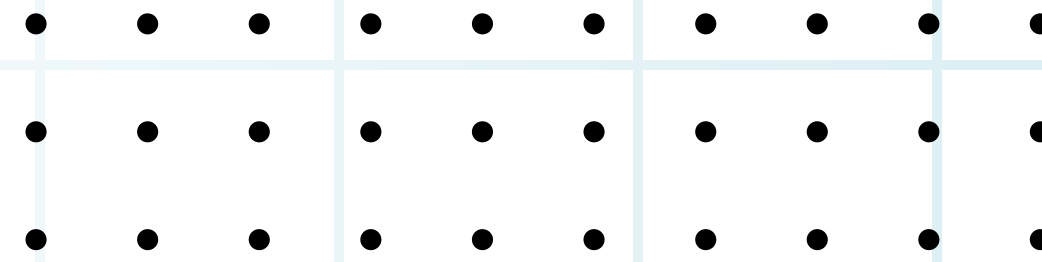
# Comandos:

## ◆ **git pull origin main**

- Sincroniza seu repositório local com o repositório remoto, trazendo as alterações mais recentes feitas por outros colaboradores.

```
PS C:\Users\enzov\Desktop\MiniCursoGit2> git pull
Already up to date.
```





**Obrigado!**